

Desafio (48h) — PetKeeper Lite

App consumidor para cadastro compartilhado de pets e controle básico de vacinas/tarefas, com foto e sincronização em tempo real. **Somente Firebase**. Execução **obrigatória no Cursor**.

O que o app faz

- **Auth**: Firebase Auth (e-mail/senha + Google).
- **Pets (Firestore)**: CRUD de pets (nome, espécie, data de nascimento, peso).
- **Foto do pet (Storage)**: upload e exibição.
- **Vacinas/Tarefas (Firestore)**: adicionar itens por pet (ex.: “V10 – 20/11/2025”, “Banho – sábado”).
- **Compartilhamento simples**: um **código de família** (string) define quem enxerga/edita os mesmos pets; usuário escolhe/entra em um código na onboarding. (Sem Dynamic Links para manter o escopo ágil.)
- **Notificação push (FCM via Cloud Function)**: botão “**Avisar família**” no detalhe do pet dispara push informando nova vacina/tarefa.
- **Sync em tempo real**: listas de pets e de vacinas/tarefas atualizam ao vivo.

Nada de backend externo, nada de FastAPI. Só Firebase (Auth, Firestore, Storage, Cloud Functions, FCM).

Integrações Firebase

- **Auth**: email/password + Google Sign-In; persistência padrão.
- **Firestore**:
 - Coleções: `users`, `families`, `pets`, `pet_tasks`.

- Consultas com `where("familyCode" == do usuário)` para isolar os dados.
 - **Storage:** pasta `pet_photos/{petId}.jpg` com validação em rules.
 - **Cloud Functions (Node 20):**
 - `notifyFamily` (**httpsCallable**): recebe `{petId, message}` → envia **FCM** a todos os tokens dos membros da família.
 - **FCM:** registrar token por usuário em `users/{uid}.fcmTokens[]`.
-

Modelo de Dados (enxuto)

```
users/{uid}
  displayName, email, familyCode, fcmTokens[]

families/{familyCode}
  createdAt, ownerId

pets/{petId}
  familyCode, name, species ("dog"|"cat"|...), birthDate, weightKg,
  photoUrl, createdAt

pet_tasks/{taskId}
  petId, type ("vaccine"|"grooming"|"other"), title, dueDate, notes,
  createdBy, createdAt, done (bool)
```

Regras de Segurança (mínimas e firmes)

Firestore

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    function hasFamily() {
```

```

        return request.auth != null && request.auth.token != null;
    }
    function userFamily() {
        return
get(/databases/${db}/documents/users/${request.auth.uid}).data.familyC
ode;
    }

    match /users/{uid} {
        allow read, write: if request.auth != null && uid ==
request.auth.uid;
    }

    match /families/{familyCode} {
        allow read, write: if request.auth != null && familyCode ==
userFamily();
    }

    match /pets/{petId} {
        allow read, write: if request.auth != null
        && resource.data.familyCode == userFamily()
        && request.resource.data.familyCode == userFamily();
    }

    match /pet_tasks/{taskId} {
        allow read, write: if request.auth != null
        && (resource.data.petId in getFamilyPetIds() ||
request.resource.data.petId in getFamilyPetIds());
    }

    function getFamilyPetIds() {
        return [];
        // (Para o desafio, valide pelo familyCode do pet recuperado via
security rules simples;
        // não é necessário lookup complexo – mantenha as writes no app
sempre com familyCode coerente.)
    }
}

```

```
}
```

Storage

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /pet_photos/{petId}.jpg {
      allow read, write: if request.auth != null; // simples para o
desafio
    }
  }
}
```

Observação: para 48h, mantenha as rules pragmáticas (checando **familyCode** em todas as writes no app). Mostre no vídeo que compreende endurecer isso depois (custom claims / rules avançadas).

Fluxos de UX

1. **Onboarding** → criar/entrar com **familyCode** (ex.: “BORGES01”).
 2. **Lista de Pets** (stream Firestore) → **Adicionar Pet** (com foto no Storage).
 3. **Detalhe do Pet** → **Adicionar Vacina/Tarefa** (stream Firestore).
 4. **Avisar Família** → botão chama **notifyFamily** → push chega em outro device logado no mesmo **familyCode**.
-

Critérios de Aceite (checklist)

- Login (e-mail/senha + Google) operacional.
- Definição de **familyCode** e isolamento dos dados por família.

- CRUD de pets + upload de foto funcionando.
 - CRUD de vacinas/tarefas com atualização em tempo real.
 - `notifyFamily` envia push para pelo menos 1 outro device real/emulador.
 - Regras Firestore/Storage impedem acesso sem Auth.
 - README explica setup Firebase, emuladores e build.
-

Entregáveis

- **Repo** com `/app` (Flutter) e `/functions` (Cloud Functions).
- **README** com:
 - Passo a passo de setup (Firebase project, `google-services.json/GoogleService-Info.plist`, Cloud Messaging, emuladores).
 - Comandos (`flutter run`, `firebase emulators:start`, `firebase deploy` opcional).
 - **Link do vídeo (≤8 min)** mostrando os 4 fluxos de UX e notificações.
 - **Seção “Como usei o Cursor”**: prompts + o que foi refatorado manualmente.

Envie o link do código (repo) e o link do vídeo para ana.flavia@equestrolabs.com

Uso do Cursor (obrigatório)

- Gerar stubs de **Widgets**, providers Riverpod, services Firebase, e a Function `notifyFamily`.

- Pedir ao candidato que **cole no README** 3–5 prompts usados e explique **por que** ajustou o código gerado (segurança, performance, testes).
 - Avaliar se a pessoa **dirige** o Cursor (e não o contrário).
-

Cronograma (48h)

Dia 1 (fundações + pets)

- Setup Flutter + Firebase (Auth, Firestore, Storage, Messaging).
- Onboarding com `familyCode`.
- Lista/Detalhe de Pets + upload de foto.
- Regras mínimas + teste manual com emuladores.

Dia 2 (tarefas + push + polimento)

- CRUD de vacinas/tarefas (streams).
- Cloud Function `notifyFamily` + FCM entre 2 devices.
- Estados de UI (loading/empty/error), pequenos testes (2 widget + 1 unit).
- Gravar vídeo (≤8 min) e finalizar README.